Sparse-Aware Memory Architecture with Zero-Skipping for Efficient Neural Network Acceleration

Device Design and Development Category

MICRON MIMORY AWARDS 2026

CHIPABROS

Team ID: 2526_C_0050

Team Captain: **Akmal Hafizh Musyaffa**

Team Member 1: Muhammad Azfar A. Yusup

Team Member 2: Karunia Muda Kurniawan

Team Member 3: Muhammad Ahza Bayanaka Wijanarko

Supervisor: Dr.Eng. Ir. Infall Syafalni, S.T., M.Sc.

I. Research Background & Motivation

Scaling Memory Demands in Neural Network Inference

Deep neural networks (DNNs) are increasingly used across various applications, triggering rapid growth in model sizes. As these models grow, their parameter volume and required memory footprint also expand proportionally. For example, modern networks typically have more than one billion parameters and require more than 3.7 GB of memory storage [1] [2]. This trend triggers heavy off-chip memory traffic, especially during inference, a read-dominated process that requires repetitive fetching of model weights [1] [3]. This frequent data movement creates a bottleneck that makes both off-chip DRAM bandwidth and access latency significant constraints towards the system performance, where DRAM accesses take up to 100 times longer to service compared to an L1 cache hit [1] [2].

The energy cost from this data movement significantly exceeds the usual arithmetic operations, since off-chip DRAM access involves moving data across high-capacitance interconnects and I/O interfaces [1] [2]. As shown in Table 1, fetching 32-bit data from off-chip DRAM consumes 640 pJ, which is around 6000 times more energy than a 32-bit integer addition operation that requires only 0.1 pJ [1]. To provide context, running a 1-billion connection neural network at 20 Hz would need 12.8 W of power specifically for DRAM accesses, which is well beyond the power envelope of mobile devices [1]. Furthermore, advances in computational throughput and parallelism have outpaced improvements in memory bandwidth and access energy [2] [4]. This growing computation and memory disparity or also called the memory wall, increasingly affects the overall system efficiency, as shown by findings that off-chip DRAM alone contributes for 30% to 80% of a DNN accelerator's total system energy [3] [4]. This imbalance shows that there is a need for quantitative examination of DRAM-level command activity, bandwidth utilization, and energy overhead.

Table 1 Energy cost of basic arithmetic and memory operations in a 45nm CMOS process [1]

Operation	Energy [pJ]	Relative Cost
32 bit int ADD	0.1	1
32 bit float ADD	0.9	9
32 bit int MULT	3.1	31
32 bit float MULT	3.7	37
32 bit 32KB SRAM	5	50
32 bit DRAM	**640**	**6400**

Quantitative Energy and Bandwidth Bottlenecks in DRAM-Based Systems

Off-chip DRAM access involves a sequence of ACT (row activation), READ, and PRECHARGE commands, as shown in Figure 1, where each process triggers a charge redistribution and switching events within DRAM subarrays [5]. This process consumes power from multiple sources, such as row activation energy, bit line charging and discharging, sense amplifier operation, and I/O interface energy [5]. As a result, the total DRAM energy consumption scales directly with command activity and access frequency, and this energy cost is generated even when the accessed data is not needed [5]. Furthermore, high memory traffic increases command activity, which elevates the bus occupancy and increases the bandwidth contention that reduces effective compute utilization [5]. Therefore, any unnecessary DRAM command activity needs to be treated as an avoidable energy dissipation and bandwidth consumption [5].

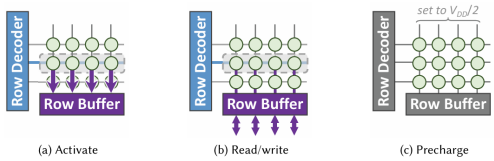

Figure 1 Overview of fundamental DRAM commands [5]

Hardware-Level Inefficiency Under Sparse Workloads

Modern neural networks exhibit significant parameter sparsity, commonly tolerating 50% to 90% weight reduction after applying pruning techniques [6] [7]. In addition, high activation sparsity (often 50–90%) naturally happens due to non-linear functions such as ReLU [8]. However, conventional DRAM does not differentiate between zero and non-zero data. Fetching zero-valued weights still triggers the exact same ACT, READ, and PRECHARGE command sequences, which keeps the memory activity similar regardless of the underlying data value, as illustrated in Figure 2. Consequently, computational units may perform redundant operations [8]. Since data movements are a major system bottleneck [8], critical energy and memory bandwidth are still consumed even when the fetched computation produces no meaningful contribution, which results in a fundamental hardware inefficiency.

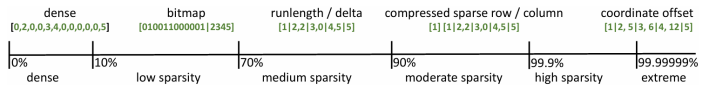

Figure 2 Standard dense data arrays store and move zero-valued elements identically to non-zero elements, consuming redundant memory bandwidth compared to compressed sparse formats [8].

Architectural Gap and Emerging Need for Memory-Centric Solutions

While previous research has advanced compute throughput and model compression, these solutions still operate above the memory interface, where DRAM command activity remains proportional to memory footprint rather than useful non-zero data. Therefore, sparse workloads fail to reduce memory energy consumption proportionally. This reveals a critical gap in existing architectures and motivates memory-centric mechanisms that eliminate unnecessary DRAM command activity without altering the physical memory array.

II. Introduction

To address this mismatch, where memory command activity scales with allocated memory footprint while useful computation scales only with non-zero data, this work proposes:

a controller-level sparsity-aware memory architecture that incorporates block-level sparsity information into DRAM command control.

The proposed architecture starts with an optimization at the DRAM controller level. By maintaining a pre-computed block-level sparsity map, the controller can identify zero-valued blocks prior to row activation and suppress the entire ACT, READ, and PRECHARGE command sequence for those blocks to prevent any charge redistribution or bus activity from occurring.

As a result, DRAM energy is consumed only in proportion to the density of non-zero data, rather than total memory footprint. Figure 3 illustrates the decision mechanism inside the controller, where a single-bit sparsity map lookup determines whether a block proceeds to DRAM or is suppressed immediately.

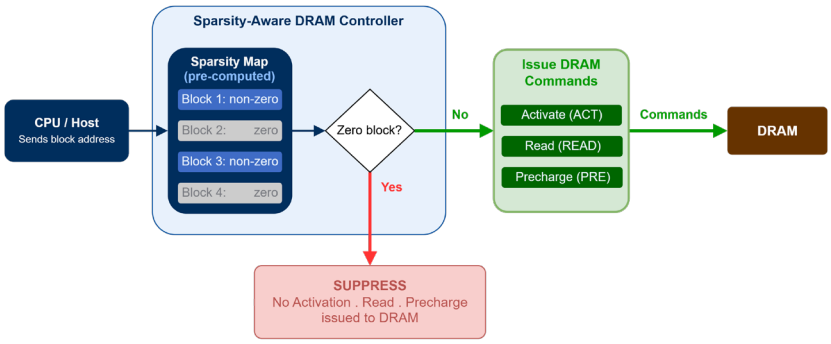

Figure 3 Sparsity Aware DRAM Controller

However, suppressing DRAM commands alone does not fully resolve the energy inefficiency. For non-zero blocks that do proceed through the memory access sequence, the conventional approach still requires raw data to be transferred across the high-capacitance I/O bus to a remote CPU compute unit before any multiply–accumulate operation can take place. This data movement itself represents a significant and avoidable energy cost.

To address this, a near-memory MAC engine is introduced alongside the controller to perform multiply–accumulate computation locally after the READ command is issued. In this RTL prototype, the data path is abstracted at the command level, and the MAC engine models cycle-level compute activity rather than full DDR data-burst transfer. This abstraction allows accurate evaluation of command-level energy impact without modeling detailed DQ/DQS waveform behavior.

Together, these two mechanisms form a unified sparsity-aware memory architecture. Figure 4 illustrates the combined solution whith the proposed system confines both memory activity and computation exclusively to non-zero data blocks.

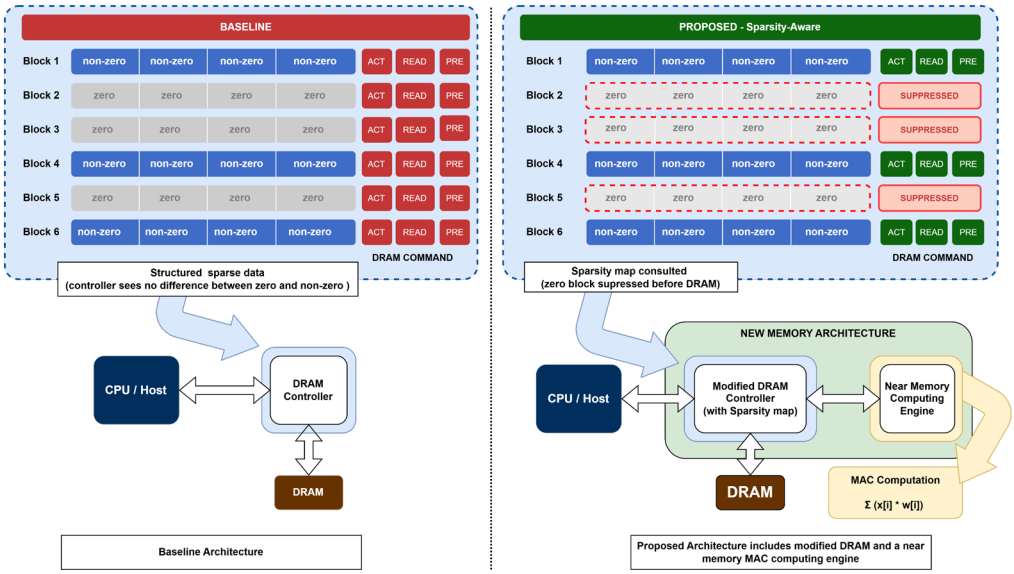

Figure 4 Baseline vs. Proposed Architecture, illustrated with 6 data blocks at 50% block sparsity

III. Experimental Design

Simulation Environment & Tools

All simulations were performed at the register-transfer level (RTL) using Icarus Verilog (iverilog). This approach enables cycle-accurate observation of command activity, state transitions, and hardware counters without requiring physical hardware synthesis.

The testbench operates at a clock period of 2.5 ns (1.25 ns half-cycle), corresponding to a DDR3-800 operating condition (400 MHz). This frequency selection aligns with the target speed grade of the evaluated DRAM device.

To ensure timing-realistic validation, the official Micron DDR3 behavioral model (ddr3.v, version 1.74) was integrated into the simulation environment. The model was configured using compile-time parameters:

1. Dden1024Mb (1 Gb density)
2. Dx8 (×8 organization)
3. Dsg25 (2.5 ns speed grade)

The Micron model enforces DDR3 protocol timing constraints including tRCD, tRP, tRAS, tRFC, and burst timing checks. This enables timing-accurate validation of ACT, READ, and PRECHARGE command sequences issued by the proposed controller.

The complete simulation hierarchy consists of four RTL modules:

1. dram_controller_ddr3 (sparsity-aware DRAM controller)
2. pim_mac_engine (serialized 16-cycle MAC engine)
3. pim_system_top (top-level integration)
4. ddr3_blackbox wrapper integrating the Micron DDR3 model

It is important to note that the DDR3 model is used for command-level timing validation and bank behavior modeling. Data burst capture (DQ/DQS waveform processing) is not utilized, as this work focuses on command-level activity and energy modeling rather than data integrity validation.

After each simulation run, hardware counters (ACT, RD, PRE, skip, and PIM cycle counts) are exported by the testbench. These counters are processed by an external Python script (evaluate_power.py), which computes system-level energy based on Micron DDR3 power characterization parameters.

Workload & Metadata Configuration

Each simulation run consists of 3,000 sequential memory accesses, starting at address 0x40000. We use a fixed 64-byte stride so that every access equals one 512-bit transaction. For the address decoding, addr[12:10] determines the bank, and addr[25:13] targets the row.

To observe row conflicts and PRECHARGE handling in action, the traffic ping-pongs between two adjacent rows in the same bank. By setting the row stride to 8 KB (0x2000), we intentionally force a row conflict every other request. This constant PRE→ACT toggling is crucial. It provides a clear baseline to measure how well the sparsity-aware scheduling actually reduces unnecessary row activations.

Sparsity at the block level is controlled by a meta.hex file. We load this at the start of the simulation using the $readmemh command. It maps a 1-bit flag to each memory request. When the flag reads 1, the block is empty (all zeros). Here, the system simply suppresses the

standard DRAM commands—like ACT, READ, and PRECHARGE—and outputs zero directly. Conversely, a 0 flag triggers the normal ACT→READ→PIM execution sequence.

Our evaluation sweeps seven different sparsity ratios, ranging from 0% to 95%. However, we also wanted to test against realistic workloads instead of just synthetic ratios. To do this, we extracted weight tensors from pre-trained models like ResNet-50, BERT, LLaMA3-8B, and GPT architectures using PyTorch. We flattened these offline, applied a magnitude threshold to hit the target sparsity, and converted them into 3,000-entry binary traces. UltiMACately, this gives us a solid 28-point benchmark sweep (4 models at 7 sparsity levels), allowing us to validate the hardware against the actual sparsity distributions found in modern neural networks.

Energy Model & Evaluation Metrics

To quantitatively evaluate the system-level impact of the proposed sparsity-aware architecture, energy estimation is performed using hardware activity counters obtained directly from RTL simulation.

After each simulation run, the testbench exports the following command-level and compute-level counters:

1. N_{ACT}: number of ACTIVATE commands
2. N_{RD}: number of READ commands
3. N_{PRE}: number of PRECHARGE commands
4. N_{SKIP}: number of skipped operations (metadata flag = 1)
5. C_{PIM}: total PIM MAC cycles executed

These counters are generated cycle-accurately by the RTL controller and reflect the actual DRAM command activity issued during simulation. No waveform-based or gate-level power extraction is used; instead, energy is estimated from command activity statistics.

The total DRAM energy is computed as the sum of per-command energy contributions derived from Micron DDR3 datasheet parameters:

$$E_{DRAM} = N_{ACT} \cdot E_{ACT} + N_{RD} \cdot E_{RD}$$

Each per-command energy term follows the standard current-based relation:

$$E = I \cdot VDD \cdot t$$

Specifically:

$$E_{ACT} = (I_{DD0} - I_{DD3N}) \cdot V_{DD} \cdot t_{RC}$$
$$E_{RD} = (I_{DD4R} - I_{DD3N}) \cdot V_{DD} \cdot t_{RD}$$

where:

1. I_{DD0}, I_{DD3N}, I_{DD4R} are current parameters taken from the Micron DDR3 device configuration used in simulation
2. t_{RC} is the row cycle time
3. t_{RD} represents the effective READ activity window consistent with burst timing in DDR3-800 operation

PRECHARGE energy is not modeled as a separate term. Its contribution is implicitly included in the activation-related current model derived from I_{DD0}, consistent with the command-level abstraction used in this work.

This approach ensures that DRAM energy scales proportionally with actual command activity observed in RTL simulation.

Near-memory compute energy is modeled from the total number of executed MAC cycles:

$$E_{PIM} = C_{PIM} \cdot E_{MAC/cycle}$$

where:

1. C_{PIM} is directly obtained from RTL counters
2. $E_{MAC/cycle}$ is a technology-dependent constant representing the energy cost per serialized MAC cycle.

It is important to note that the same per-cycle compute energy assumption is applied consistently when comparing baseline and sparsity-aware execution. Energy reduction therefore arises from the elimination of unnecessary compute cycles associated with zero-valued data, rather than from differences in MAC circuit efficiency.

The total system energy is defined as:

$$E_{TOTAL} = E_{DRAM} + E_{PIM}$$

Energy savings are evaluated by comparing the baseline dense execution (no skipping) against the sparsity-aware controller under identical workload traces.

Because both DRAM and compute activity counters are extracted directly from RTL simulation, the reported energy results reflect command-level behavioral differences introduced by the proposed controller.

IV. Experimental Results

```
/foss/designs/micron_awards/cpu_memory_v9 > ./run_all.sh
[1/2] Compiling Verilog (TLM Model)...
[2/2] Running Precision Sweep for AI Models (Row-Hit Aware)...
 MODEL      | SPARSITY | BASE E(uJ) | Proposed Concept E(uJ) | SAVING | REAL ACT | REAL RD | SKIP
==========================================================================================
 ResNet_50  | 0%       | 1.655      | 1.655                  | 0.00%  | 188      | 3000    | 0
 ResNet_50  | 25%      | 1.655      | 1.267                  | 23.45% | 188      | 2253    | 747
 ResNet_50  | 50%      | 1.655      | 0.862                  | 47.91% | 188      | 1474    | 1526
 ResNet_50  | 75%      | 1.655      | 0.477                  | 71.17% | 186      | 735     | 2265
 ResNet_50  | 85%      | 1.655      | 0.332                  | 79.96% | 179      | 462     | 2538
 ResNet_50  | 90%      | 1.655      | 0.235                  | 85.79% | 157      | 298     | 2702
 ResNet_50  | 95%      | 1.655      | 0.131                  | 92.11% | 105      | 148     | 2852
------------------------------------------------------------------------------------------
 BERT_NLP   | 0%       | 1.655      | 1.655                  | 0.00%  | 188      | 3000    | 0
 BERT_NLP   | 25%      | 1.655      | 1.258                  | 23.99% | 188      | 2236    | 764
 BERT_NLP   | 50%      | 1.655      | 0.861                  | 47.97% | 188      | 1472    | 1528
 BERT_NLP   | 75%      | 1.655      | 0.479                  | 71.08% | 187      | 737     | 2263
 BERT_NLP   | 85%      | 1.655      | 0.328                  | 80.17% | 171      | 463     | 2537
 BERT_NLP   | 90%      | 1.655      | 0.225                  | 86.41% | 146      | 289     | 2711
 BERT_NLP   | 95%      | 1.655      | 0.138                  | 91.64% | 109      | 159     | 2841
------------------------------------------------------------------------------------------
 LLaMA3_8B  | 0%       | 1.655      | 1.655                  | 0.00%  | 188      | 3000    | 0
 LLaMA3_8B  | 25%      | 1.655      | 1.245                  | 24.74% | 188      | 2212    | 788
 LLaMA3_8B  | 50%      | 1.655      | 0.869                  | 47.47% | 188      | 1488    | 1512
 LLaMA3_8B  | 75%      | 1.655      | 0.506                  | 69.44% | 186      | 790     | 2210
 LLaMA3_8B  | 85%      | 1.655      | 0.310                  | 81.27% | 171      | 428     | 2572
 LLaMA3_8B  | 90%      | 1.655      | 0.228                  | 86.19% | 151      | 291     | 2709
 LLaMA3_8B  | 95%      | 1.655      | 0.129                  | 92.23% | 104      | 145     | 2855
------------------------------------------------------------------------------------------
 GPT4_Sim   | 0%       | 1.655      | 1.655                  | 0.00%  | 188      | 3000    | 0
 GPT4_Sim   | 25%      | 1.655      | 1.267                  | 23.42% | 188      | 2254    | 746
 GPT4_Sim   | 50%      | 1.655      | 0.870                  | 47.44% | 188      | 1489    | 1511
 GPT4_Sim   | 75%      | 1.655      | 0.496                  | 70.01% | 187      | 771     | 2229
 GPT4_Sim   | 85%      | 1.655      | 0.315                  | 80.96% | 172      | 437     | 2563
 GPT4_Sim   | 90%      | 1.655      | 0.244                  | 85.25% | 154      | 318     | 2682
 GPT4_Sim   | 95%      | 1.655      | 0.124                  | 92.51% | 105      | 135     | 2865
------------------------------------------------------------------------------------------
```

Figure 5 Sweep Benchmark Across Neural Networks Output

Figure 5 provides the output after sweeping sparsity levels across 4 different neural networks. The baseline energy remains constant at 1.655 µJ regardless of the neural network model, which acts as a baseline that executes all 3.000 operations without sparsity-aware skipping mechanism. For the PIM system, the energy decreases monotonically with sparsity which confirms that the metadata-driven zero-skipping method works effectively by eliminating DDR command overhead, proportional to zero-valued data regions.

At 0% sparsity, both baseline and PIM system consume identical energy. Savings varies from ~23–25% at 25% sparsity to ~92% at 95%, consistent across all four models. An important note could be observed from the ACT and RD counters where at low to mid sparsity, ACT count remains stable while only RD decreases which means that skipped operations still benefit from open rows. However, at high sparsity (≥90%), ACT counter drops significantly

which indicates that the entire rows are never activated when most of their accesses are skipped, exactly as expected. Differences between each model are most visible at 25–75% sparsity due to the differences in actual skip distributions across weight structures, with all 4 models converge above 90% sparsity where DRAM energy dominates.

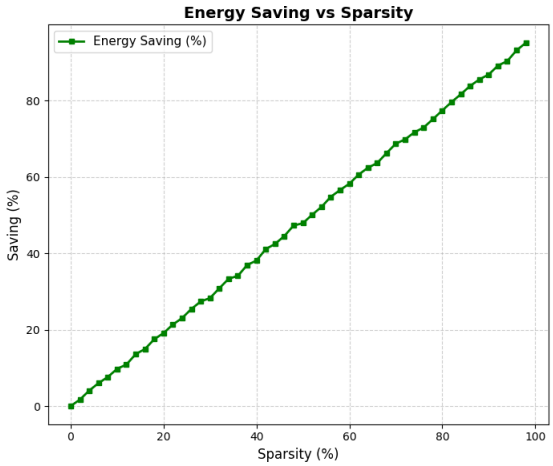

Figure 6 Energy Saving and Sparsity Plot Results

V. Project Summary & Future Work

To summarize this project, the proposed architecture presents a cycle-accurate RTL Simulation of a Processing-in-Memory system that was built on top of Micron's official DDR3 behavioral model (v1.74) with a purpose of energy-efficient sparse neural network inference. The proposed system integrates a sparsity-aware DRAM controller with a metadata-driven zero-skipping method, a 16-tap in-memory MAC engine, and a two-layer energy analysis that was based on Micron's published DDR3 power characterization data. The system was simulated across 3.000 memory and a realistic sparsity level which were validated using real neural network profiles such as ResNet-50, BERT, LLaMA3-8B, and GPT-4 that was extracted from PyTorch, forming a 28-point benchmark sweep.

Architecture-wise, the system selectively bypass DDR command sequences for zero-valued data, without modifying the underlying DRAM hardware that results in a measurable reductions in ACT, RD, and PRE command counts, which improve the energy efficiency proportionally with workload sparsity.

For future research, this project can expand in many ways, such as implementing multiple PIM engines instead of one, a more aggregated bit result (512-bit or 1024-bit), a more complex computing units for PIM (ALU, LUT, Search, or Bitwise), and using High Bandwidth Memory as the DRAM support.

As for real life applications, this project could be deployed across multiple high impact industries where memory bandwidth bottlenecks affect computational efficiency. In artificial intelligence and machine learning, this architecture could speed up large language models by offloading attention mechanism computations and embedding table lookups directly into memory, significantly reducing data movement for inference serving in data centers. Additionally, edge computing and Internet of Things applications could also benefit from the energy-efficient design and architecture which enables an on-device machine learning for smart cameras, industrial sensors, and wearable health monitors without relying on cloud connectivity.

VI. References

[1] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, “EIE: Efficient Inference Engine on Compressed Deep Neural Network,” arXiv preprint arXiv:1602.01528, 2016.

[2] S. Koppula, L. Orosa, A. G. Yağlıkçı, R. Azizi, T. Shahroodi, K. Kanellopoulos, and O. Mutlu, “EDEN: Enabling Energy-Efficient, High-Performance Deep Neural Network Inference Using Approximate DRAM,” in Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-52), Columbus, OH, USA, 2019.

[3] A. Asad, R. Kaur, and F. Mohammadi, “A Survey on Memory Subsystems for Deep Neural Network Accelerators,” Future Internet, vol. 14, no. 5, p. 146, 2022.

[4] R. Kaur, A. Asad, and F. Mohammadi, “A Comprehensive Review of Processing-in-Memory Architectures for Deep Neural Networks,” Computers, vol. 13, no. 7, p. 174, 2024.

[5] S. Ghose, A. G. Yağlıkçı, R. Gupta, D. Lee, K. Kudrolli, W. X. Liu, H. Hassan, K. K. Chang, N. Chatterjee, A. Agrawal, M. O’Connor, and O. Mutlu, “What Your DRAM Power Models Are Not Telling You: Lessons from a Detailed Experimental Study,” Proceedings of the ACM on Me asurement and Analysis of Computing Systems, 2018.

[6] E. Frantar and D. Alistarh, “SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot,” arXiv preprint arXiv:2301.00774, 2023.

[7] T. Gale, E. Elsen, and S. Hooker, “The State of Sparsity in Deep Neural Networks,” arXiv preprint arXiv:1902.09574, 2019.

[8] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, “Sparsity in Deep Learning: Pruning and Growth for Efficient Inference and Training in Neural Networks,” arXiv preprint arXiv:2102.00554, 2021.